

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №6**

«Автоматизоване тестування: pytest, fixtures, mocking та coverage»

з курсу «Професійний Python»

Варіант №1 — «Тестування системи обліку студентів»

Виконав: Лис Артур

Група: ФЕП-31

Перевірив(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Навчитися писати автоматизовані тести для власного проєкту: unit- та інтеграційні тести на `pytest`, фікстури з різними `score`, параметризацію, перевірку винятків і чисел з плаваючою комою, ізоляцію через `tmp_path` і `monkeypatch`, тест-дублери `Mock/AsyncMock`, а також вимірювати `statement-` і `branch-coverage` та тримати його вище порогу.

2. Індивідуальний варіант і постановка задачі

Варіант 1 — тестування системи обліку студентів, яку я розвиваю з ЛР1 (пакет `student_manager`). Обов'язковий перелік: створення студента, валідація оцінки з межами 0 і 100, некоректна оцінка, середній бал, порожня колекція, фільтр за групою, найкращий студент, імпорт CSV, некоректний CSV, експорт JSON, конфігурація, репозиторій, `Mock` для нотифікатора та асинхронний сервіс через `AsyncMock`.

Вимоги до набору: щонайменше 15 unit- і 3 інтеграційні тести, 3 фікстури, 2 параметризовані тести, `pytest.raises` із власним винятком, `pytest.approx`, `tmp_path`, `monkeypatch`, `return_value`, `side_effect`, перевірка взаємодії, `coverage ≥ 80 %` з увімкненими гілками, поділ `tests/` на `conftest.py`, `unit/` та `integration/`.

Що зроблено понад мінімум: фабрична фікстура, параметризована фікстура (`policy`), `autouse`-фікстура з `yield`, `create_autospec` для протоколу, власні маркери `integration` та `slow` із `--strict-markers`, поріг покриття у `pyproject.toml`, HTML-звіт. Разом 175 тестів (57 нових), покриття 93 %.

3. Тестований проєкт і нові «шви»

Проєкт — той самий `student-manager`: модель `Student` та `StudentRegistry` (ЛР1), аналітика (ЛР2), потокова обробка (ЛР3), доменна модель із контейнером і `Notifier` (ЛР4), пакет `storage` з імпортом, експортом і YAML-конфігурацією (ЛР5). Тести попередніх робіт (118) перенесено в `tests/unit/` без змін.

Щоб дублери мали що підмінити, у продакшн-код додано три невеликі модулі. `clock.now` — єдине місце, де проєкт питає час (його підміняє `monkeypatch`); `snapshot.registry_snapshot` формує датований підсумок реєстру; пакет `notify` містить `DigestSender`, що надсилає листи через протокол `Mailer`, і `AsyncStudentService`, що працює з асинхронним шлюзом `StudentGateway`. Обидва протоколи в тестах замінено на `Mock` та `AsyncMock`.

```
def _system_now() -> datetime:
    return datetime.now(tz=timezone.utc)

now: Callable[[], datetime] = _system_now

class Mailer(Protocol):
    def send(self, recipient: str, subject: str, body: str) -> str:
        """Return the message id assigned by the transport."""
```

...

```
class DigestError(StudentManagerError):
    """The transport refused a message; the digest reports which group it was."""

    def send_all(self, registry: StudentRegistry) -> dict[str, str]:
        """Send one digest per group; return {group: message id}. A transport failure stops the run."""
        sent: dict[str, str] = {}
        for group in registry.groups():
            subject, body = self.compose(registry, group)
            try:
                sent[group] = self._mailer.send(self.recipient_for(group), subject, body)
            except OSError as error:
                raise DigestError(f"не вдалося надіслати дайджест групі {group}: {error}") from error
        return sent

class StudentGateway(Protocol):
    async def fetch(self, student_id: int) -> Student | None: ...

    async def save(self, student_id: int, student: Student) -> None: ...

    async def require(self, student_id: int) -> Student:
        student = await self._gateway.fetch(student_id)
        if student is None:
            raise StudentNotFoundError(f"студента з id={student_id} не знайдено")
        return student

    async def average_of(self, student_ids: Iterable[int]) -> float:
        """All fetches run concurrently; an empty id list is an error, not a silent 0.0."""
        ids = list(student_ids)
        if not ids:
            raise ValueError("порожній список студентів")
        students = await asyncio.gather(*(self.require(i) for i in ids))
        return sum(s.average_grade for s in students) / len(students)
```

4. Структура test suite і тестова матриця

```
└─ tests
    ├── conftest.py
    ├── integration
    │   ├── test_container.py
    │   ├── test_csv_roundtrip.py
    │   └── test_pipeline.py
    └── unit
        ├── test_analytics.py
        ├── test_config.py
        ├── test_domain.py
        ├── test_models.py
        ├── test_notify.py
        ├── test_registry.py
        ├── test_snapshot.py
        ├── test_storage.py
        └── test_streaming.py
```

tests/unit/ — по модулю на пакет, без файлової системи та мережі; tests/integration/ — кілька модулів разом на справжніх файлах у tmp_path (маркер integration ставиться автоматично за шляхом у pytest_collection_modifyitems). Матриця нижче показує, який тест закриває який пункт варіанта.

Пункт варіанта	Тест	Техніка
Створення студента	test_models::test_student_creation_normalises_names	фабрична фікстура
Межі 0 і 100	test_boundary_grades_are_accepted	parametrize + ids
Некоректна оцінка	test_invalid_grade_raises_custom_exception	raises + exc_info, nan
Середній бал	test_average_is_compared_with_approx	pytest.approx
Порожня колекція	test_empty_registry	raises StudentNotFoundError
Фільтр за групою	test_filter_by_group	parametrize (3 випадки)
Найкращий студент	test_best_student_overall_and_per_group	state testing
Імпорт CSV / експорт JSON	integration::test_csv_to_registry_to_json_and_back	tmp_path, round-trip
Некоректний CSV	test_malformed_rows_follow_the_policy	параметризована фікстура policy
Конфігурація	test_config (4 тести)	monkeypatch.setenv/delenv
Репозиторій, контейнер	integration::test_container (3 тести)	Mock(spec=Notifier)
Нотифікатор	test_notify::test_digest_* (6 тестів)	return_value, side_effect, autospec
Async-сервіс	test_notify::test_*gateway* (5 тестів)	AsyncMock, assert_awaited_*
Детермінований час	test_snapshot (3 тести)	monkeypatch.setattr(clock)

5. Фікстури (conftest.py)

Фікстури з `conftest.py` не імпортують: `pytest` сам підставляє їх за іменем параметра. Тут п'ять різних видів: `session-scope` дані, `function-scope` реєстр, фабрика, обгортка над `tmp_path`, підміна часу, параметризована політика імпорту та `autouse`-сторож із `yield`.

```
@pytest.fixture(scope="session")
def sample_students() -> tuple[Student, ...]:
    return (
        Student("Марта", "Гнатишин", "ФЕП-31с", 93.4),
        Student("Остан", "Дзюба", "ФЕП-31с", 78.9),
        Student("Соломія", "Кравець", "ФЕП-32", 88.1),
    )

@pytest.fixture
def registry(sample_students: tuple[Student, ...]) -> StudentRegistry:
    return StudentRegistry(sample_students)

@pytest.fixture
def make_student() -> StudentFactory:
    """Factory fixture: each call gives a valid student; keyword arguments override single fields."""
    numbers = count(1)

    def factory(**overrides: object) -> Student:
        fields: dict[str, object] = {
            "first_name": "Тест", "last_name": f"Студент{next(numbers)}", "group": "ФЕП-31с",
            "average_grade": 75.0,
        }
        fields.update(overrides)
        return Student(**fields) # type: ignore[arg-type]

    return factory

@pytest.fixture
def write_csv(tmp_path: Path) -> CsvWriter:
    """Write a variant-1 students CSV into the test's private temporary directory."""
    def writer(rows: Sequence[Sequence[object]], name: str = "students.csv") -> Path:
        lines = [",".join(STUDENT_FIELDS)] + [",".join(str(v) for v in row) for row in rows]
        path = tmp_path / name
        path.write_text("\n".join(lines) + "\n", encoding="utf-8")
        return path

    return writer

@pytest.fixture
def frozen_clock(monkeypatch: pytest.MonkeyPatch) -> datetime:
    """Deterministic time: production code reads ``clock.now``; monkeypatch restores it after the test."""
    monkeypatch.setattr(clock, "now", lambda: FROZEN_TIME)
    return FROZEN_TIME
```

```

@pytest.fixture(params=[STRICT, TOLERANT], ids=["strict", "tolerant"])
def policy(request: pytest.FixtureRequest) -> ImportPolicy:
    """Parametrised fixture: every test that asks for ``policy`` runs twice."""
    result: ImportPolicy = request.param
    return result

@pytest.fixture(autouse=True)
def isolation_guard() -> Iterator[None]:
    """Autouse: runs around every test without being requested. Fails loudly if a test leaked global
    state
    (a patched clock or a stray handler on the package logger), so the leak is blamed on the right
    test."""
    log = logging.getLogger("student_manager")
    handlers_before = list(log.handlers)
    yield
    assert clock.now is clock._system_now, "тест залишив підмінений clock.now"
    assert log.handlers == handlers_before, "тест залишив обробник логера student_manager"

```

Session-scope `sample_students` створюється один раз, а `registry` — на кожен тест, бо тести додають у нього студентів (`test_adding_a_student_changes_the_best`). Код після `yield` в `isolation_guard` — це `teardown`: він виконується навіть коли тест упав, і перевіряє, що тест не залишив після себе підмінений `clock.now` чи зайвий обробник логера.

6. Unit-тести: межі, винятки, `approx`, порожня колекція

```

@pytest.mark.parametrize("grade", [0, 0.0, 100, 100.0, 59.999], ids=["0", "0.0", "100", "100.0",
"59.999"])
def test_boundary_grades_are_accepted(make_student: StudentFactory, grade: float) -> None:
    assert make_student(average_grade=grade).average_grade == grade

@pytest.mark.parametrize(("grade", "excellent"), [(89.99, False), (90.0, True), (100, True)])
def test_is_excellent_starts_at_ninety(make_student: StudentFactory, grade: float, excellent: bool) ->
None:
    assert make_student(average_grade=grade).is_excellent is excellent

@pytest.mark.parametrize("grade", [-0.01, -1, 100.01, 101, math.nan], ids=["-0.01", "-1", "100.01",
"101", "nan"])
def test_invalid_grade_raises_custom_exception(make_student: StudentFactory, grade: float) -> None:
    with pytest.raises(InvalidGradeError) as info:
        make_student(average_grade=grade)
    assert isinstance(info.value, StudentManagerError)
    assert isinstance(info.value, ValueError)

```

Для діапазону 0–100 перевірено обидві межі в цілому та дробовому вигляді, значення одразу за межами (-0.01 , 100.01) і `nan`, який не проходить жодне порівняння. `exc_info.value` дає сам об'єкт винятку: `InvalidGradeError` одночасно є і `StudentManagerError`, і `ValueError`, тож старий код із `except ValueError` не ламається.

```

def test_average_is_compared_with_approx(make_student: StudentFactory) -> None:
    registry = StudentRegistry(make_student(average_grade=g) for g in (70.1, 70.2, 70.3))

```

```
assert registry.group_average("ФЕП-31c") == pytest.approx(70.2)
assert registry.group_average("ФЕП-31c") == pytest.approx(70.2, abs=1e-9)
```

```
def test_empty_registry(registry: StudentRegistry) -> None:
    empty = StudentRegistry()
    assert len(empty) == 0
    assert list(empty) == []
    assert empty.groups() == []
    assert len(registry) == 3 # the shared fixture is untouched
    with pytest.raises(StudentManagerError):
        empty.best()
```

Сума $70.1 + 70.2 + 70.3$ у двійковій арифметиці не дорівнює 210.6 точно, тому `==` для середнього ненадійний; `pytest.approx` за замовчуванням допускає відносну похибку 10^{-6} , а другий `assert` показує явний абсолютний допуск. `test_empty_registry` заодно доводить, що спільна фікстура не змінилася.

7. tmp_path і monkeypatch

`tmp_path` — унікальний каталог на кожен тест, який `pytest` сам створює і прибирає; конфігураційний `YAML` і `CSV` пишуться туди, тож тести не бачать один одного. `monkeypatch.setenv/delenv` змінюють справжній `os.environ`, а після тесту повертають його назад — наступний тест перевіряє саме це.

```
@pytest.fixture
def config_path(tmp_path: Path) -> Path:
    write_students_csv(tmp_path / "students.csv")
    (tmp_path / "output").mkdir()
    return write_demo_config(tmp_path / "config.yaml")

def test_environment_variable_beats_the_file(config_path: Path, monkeypatch: pytest.MonkeyPatch) ->
None:
    monkeypatch.setenv("STUDENT_MANAGER_MINIMUM_GRADE", "90")
    monkeypatch.setenv("STUDENT_MANAGER_SKIP_INVALID", "false")
    config = load_config(config_path)
    assert config.processing.minimum_grade == 90
    assert config.processing.skip_invalid is False

def test_environment_is_clean_again_in_the_next_test(config_path: Path, monkeypatch:
pytest.MonkeyPatch) -> None:
    monkeypatch.delenv("STUDENT_MANAGER_MINIMUM_GRADE", raising=False)
    config = load_config(config_path)
    assert config.processing.minimum_grade == 80 # the value from the YAML file
    assert config.processing.skip_invalid is True

@pytest.mark.parametrize("value", ["150", "-1", "abc"])
def test_invalid_override_is_a_configuration_error(config_path: Path, monkeypatch: pytest.MonkeyPatch,
value: str) -> None:
```

```
monkeypatch.setenv("STUDENT_MANAGER_MINIMUM_GRADE", value)
with pytest.raises(ConfigurationError):
    load_config(config_path)
```

Підміна функції: `monkeypatch.setattr(clock, "now", lambda: FROZEN_TIME)` у фікстурі `frozen_clock`. Патчити треба там, де ім'я **використовується**: `snapshot.py` звертається до `clock.now` через модуль, а не імпортує функцію напряму, інакше підміна атрибута в `clock` нічого б не змінила.

```
def test_snapshot_uses_injected_clock(registry: StudentRegistry, frozen_clock: datetime) -> None:
    snapshot = registry_snapshot(registry)
    assert snapshot["generated_at"] == "2026-09-08T10:00:00+00:00"
    assert snapshot["total"] == 3
    assert snapshot["groups"]["ФЕП-31с"] == pytest.approx(86.2, abs=0.05)
    assert snapshot["excellent"] == ["Гнатишин Марта"]
```

8. Mock: return_value, side_effect, autospec, interaction assertions

`mailer` створено через `create_autospec(Mailer, instance=True)`: дублер знає сигнатуру `send` і падає з `TypeError` на неправильних аргументах, чого звичайний `Mock()` не робить. Перший тест перевіряє і стан (повернутий словник), і взаємодію (`call_count`, `assert_any_call`).

```
def test_digest_sends_one_message_per_group(registry: StudentRegistry, mailer: Mailer) -> None:
    mailer.send.return_value = "msg-1" # type: ignore[attr-defined]
    sent = DigestSender(mailer).send_all(registry)
    assert sent == {"ФЕП-31с": "msg-1", "ФЕП-32": "msg-1"}
    assert mailer.send.call_count == 2 # type: ignore[attr-defined]
    mailer.send.assert_any_call( # type: ignore[attr-defined]
        "феп-31с@lnu.edu.ua", "Дайджест групи ФЕП-31с",
        "Середній бал: 86.2. Найкращий результат: Гнатишин Марта (93.4).",
    )
```

```
def test_message_ids_come_from_side_effect_sequence(registry: StudentRegistry, mailer: Mailer) -> None:
    mailer.send.side_effect = ["a", "b"] # type: ignore[attr-defined]
    assert DigestSender(mailer).send_all(registry) == {"ФЕП-31с": "a", "ФЕП-32": "b"}
```

```
def test_transport_failure_is_wrapped_into_domain_error(registry: StudentRegistry, mailer: Mailer) -> None:
    mailer.send.side_effect = ["ok", ConnectionError("smtp down")] # type: ignore[attr-defined]
    with pytest.raises(DigestError, match="ФЕП-32") as info:
        DigestSender(mailer).send_all(registry)
    assert isinstance(info.value.__cause__, ConnectionError)
```

```
def test_autospec_rejects_a_wrong_signature(mailer: Mailer) -> None:
    with pytest.raises(TypeError):
        mailer.send("only-recipient") # type: ignore[call-arg]
```

```
def test_empty_registry_sends_nothing(mailer: Mailer) -> None:
    assert DigestSender(mailer).send_all(StudentRegistry()) == {}
    mailer.send.assert_not_called() # type: ignore[attr-defined]
```

`return_value` — одна й та сама відповідь на кожен виклик; `side_effect` зі списком — послідовність відповідей, а елемент-виняток кидається замість повернення. Так одним дублером перевірено і щасливий шлях, і згорання `ConnectionError` у доменний `DigestError` зі збереженим `__cause__`.

9. AsyncMock

`AsyncMock(spec=StudentGateway)` робить `fetch` і `save` awaitable-дублерами. Корутини запускаються через `asyncio.run` у звичайному синхронному тесті — окремий плагін не потрібен, а цикл подій створюється і закривається в межах тесту. Перевіряються `await_count`, `assert_has_awaits`, `assert_awaited_once_with` і `assert_not_awaited`.

```
@pytest.fixture
def gateway() -> AsyncMock:
    """spec=Protocol: the async methods of the protocol become AsyncMocks, anything else is an
    AttributeError."""
    return AsyncMock(spec=StudentGateway)

def test_average_awaits_every_student(gateway: AsyncMock, make_student: StudentFactory) -> None:
    gateway.fetch.side_effect = [make_student(average_grade=80.0), make_student(average_grade=90.0)]
    service = AsyncStudentService(gateway)
    assert asyncio.run(service.average_of([1, 2])) == pytest.approx(85.0)
    assert gateway.fetch.await_count == 2
    gateway.fetch.assert_has_awaits([call(1), call(2)], any_order=True)

def test_missing_student_raises_and_is_awaited_once(gateway: AsyncMock) -> None:
    gateway.fetch.return_value = None
    with pytest.raises(StudentNotFoundError, match="id=42"):
        asyncio.run(AsyncStudentService(gateway).require(42))
    gateway.fetch.assert_awaited_once_with(42)

def test_empty_id_list_never_touches_the_gateway(gateway: AsyncMock) -> None:
    with pytest.raises(ValueError):
        asyncio.run(AsyncStudentService(gateway).average_of([]))
    gateway.fetch.assert_not_awaited()

def test_bump_grade_caps_at_hundred_and_saves(gateway: AsyncMock, make_student: StudentFactory) ->
None:
    gateway.fetch.return_value = make_student(average_grade=97.5)
    updated = asyncio.run(AsyncStudentService(gateway).bump_grade(7, 5.0))
    assert updated.average_grade == 100.0
    gateway.save.assert_awaited_once_with(7, updated)
```

10. Інтеграційні тести

Перший тест проганяє справжній CSV через імпорт ЛР5, реєстр ЛР1, експорт у JSON і зворотний імпорт: рядки після круга мають дорівнювати вихідним. Другий використовує

параметризовану фікстуру policy, тому виконується двічі — у strict-режимі імпорт падає з `DataError`, у tolerant пропускає зіпсовані рядки.

```
def test_csv_to_registry_to_json_and_back(write_csv: CsvWriter, tmp_path: Path) -> None:
    source = write_csv(VALID_ROWS)
    statistics = ImportStatistics()
    rows = list(import_students(source, policy=STRICT, statistics=statistics))
    assert statistics.valid == 3

    registry = StudentRegistry(row.to_student() for row in rows)
    assert registry.best().full_name == "Гнатишин Марта"
    assert registry.group_average("ФЕП-31с") == pytest.approx(86.15)

    target = tmp_path / "export.json"
    assert write_rows((row.to_dict() for row in rows), target, fieldnames=STUDENT_FIELDS) == 3
    payload = json.loads(target.read_text(encoding="utf-8"))
    assert [item["name"] for item in payload] == ["Гнатишин Марта", "Дзюба Остап", "Кравець Соломія"]

    again = list(import_students(target, policy=STRICT, statistics=ImportStatistics()))
    assert again == rows

def test_strict_run_aborts_and_leaves_no_half_written_output(workspace: Path) -> None:
    config = load_config(write_demo_config(workspace / "config.yaml", skip_invalid=False))
    with pytest.raises(DataError):
        run_pipeline(config)
    assert not (workspace / "output" / "students.json").exists()
```

Контейнер ЛР4 збирається з `Mock(сpec=Notifier)` замість консольного нотифікатора: після двох зарахувань, двох записів на курс і однієї незадовільної оцінки має бути рівно п'ять повідомлень, а останнє — адресоване студентові з оцінкою 35.

```
def test_admission_enrollment_and_a_failing_grade_notify(app: Container, notifier: Mock) -> None:
    marta = admit(app, 1, "Марта", "Гнатишин", "marta@lnu.edu.ua")
    ostap = admit(app, 2, "Остап", "Дзюба", "ostap@lnu.edu.ua")
    app.grading.grade(1, 10, 95)
    app.grading.grade(2, 10, 35)

    assert notifier.send.call_count == 5 # 2 admissions + 2 enrollments + 1 failing grade
    recipient, message = notifier.send.call_args.args
    assert recipient == ostap.contact
    assert "Незадовільна" in message and "Професійний Python" in message
    assert app.grading.average_of(1) == pytest.approx(95.0)
    assert app.reporting.ranking()[0][0] is marta
```

11. Результати запуску

11.1. pytest -q, маркери та collect-only

```
$ pytest -q
..... [ 82%]
..... [100%]
175 passed in 1.10s

$ pytest -q -m integration
```

```

..... [100%]
10 passed, 165 deselected in 0.32s

$ pytest -q -m "not slow"
..... [100%]
174 passed, 1 deselected in 1.09s

$ pytest --collect-only -q | tail -1
175 tests collected in 0.15s

```

11.2. pytest -v: ідентифікатори параметризованих тестів

```

tests/unit/test_models.py::test_boundary_grades_are_accepted[0] PASSED
tests/unit/test_models.py::test_boundary_grades_are_accepted[0.0] PASSED
tests/unit/test_models.py::test_boundary_grades_are_accepted[100] PASSED
tests/unit/test_models.py::test_boundary_grades_are_accepted[100.0] PASSED
tests/unit/test_models.py::test_boundary_grades_are_accepted[59.999] PASSED
tests/unit/test_models.py::test_invalid_grade_raises_custom_exception[-0.01] PASSED
tests/unit/test_models.py::test_invalid_grade_raises_custom_exception[-1] PASSED
tests/unit/test_models.py::test_invalid_grade_raises_custom_exception[100.01] PASSED
tests/unit/test_models.py::test_invalid_grade_raises_custom_exception[101] PASSED
tests/unit/test_models.py::test_invalid_grade_raises_custom_exception[nan] PASSED
tests/unit/test_notify.py::test_average_awaits_every_student PASSED
tests/unit/test_notify.py::test_missing_student_raises_and_is_awaited_once PASSED
tests/unit/test_notify.py::test_empty_id_list_never_touches_the_gateway PASSED
tests/unit/test_notify.py::test_bump_grade_caps_at_hundred_and_saves PASSED

```

11.3. Які фікстури отримує тест (--fixtures-per-test)

```

frozen_clock -- tests\conftest.py:81
    Deterministic time: production code reads `clock.now`; monkeypatch restores it after the test.
isolation_guard -- tests\conftest.py:102
    Autouse: runs around every test without being requested. Fails loudly if a test leaked global state
    (a patched clock or a stray handler on the package logger), so the leak is blamed on the right
    test.
--
registry -- tests\conftest.py:49
    no docstring available
sample_students -- tests\conftest.py:40
    no docstring available

```

12. Coverage

Команда: `pytest --cov=student_manager --cov-branch --cov-report=term-missing --cov-report=html`. Демонстраційні `__main__.py` та `benchmark.py` виключено в `[tool.coverage.run] omit` — це сценарії для звітів, а не логіка. Нижче фрагмент таблиці (колонки Branch/BrPart — гілки та частково пройдені гілки).

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src\student_manager\clock.py	6	0	0	0	100%	
src\student_manager\domain\services.py	85	3	12	2	95%	43, 52, 154
src\student_manager\notify__init__.py	3	0	0	0	100%	
src\student_manager\notify\async_service.py	26	0	4	0	100%	
src\student_manager\notify\digest.py	28	0	2	0	100%	
src\student_manager\registry.py	55	8	16	4	80%	34, 53, 66-67, 68-71, 71->73, 76-79
src\student_manager\snapshot.py	11	0	0	0	100%	

src\student_manager\storage\exporters.py	109	15	26	2	87%	37, 67, 86-89, 113-116, 135-139
src\student_manager\storage\services.py	107	0	10	1	99%	174->176
src\student_manager\streaming\experiment.py	108	26	8	0	71%	63-67, 102-110, 114, 145, 149-153, 157, 161-168
TOTAL	1834	102	280	25	93%	

Required test coverage of 80.0% reached. Total coverage: 93.42%

12.1. Фрагмент HTML-звіту

```
htmlcov/index.html – 45 файлів; фрагмент таблиці:
src\student_manager\__init__.py | &nbsp; | 100% | 1 | 0 | 0 | &nbsp; | 100% | 0 | 0 | &nbsp; | 100%
src\student_manager\analytics\analytics.py | &nbsp; | 87% | 45 | 6 | 0 | &nbsp; | 100% | 2 | 0 |
&nbsp; | 87%
src\student_manager\analytics\data.py | &nbsp; | 100% | 11 | 0 | 0 | &nbsp; | 100% | 0 | 0 | &nbsp; |
100%
src\student_manager\analytics\decorators.py | &nbsp; | 100% | 28 | 0 | 0 | &nbsp; | 100% | 0 | 0 |
&nbsp; | 100%
src\student_manager\analytics\processors.py | &nbsp; | 90% | 29 | 3 | 0 | &nbsp; | 100% | 2 | 0 |
&nbsp; | 90%
src\student_manager\clock.py | &nbsp; | 100% | 6 | 0 | 0 | &nbsp; | 100% | 0 | 0 | &nbsp; | 100%
src\student_manager\streaming\pipeline.py | &nbsp; | 100% | 37 | 0 | 0 | &nbsp; | 100% | 4 | 0 |
&nbsp; | 100%
src\student_manager\streaming\readers.py | &nbsp; | 98% | 41 | 1 | 0 | &nbsp; | 88% | 8 | 1 | &nbsp; |
96%
src\student_manager\streaming\transforms.py | &nbsp; | 100% | 13 | 0 | 0 | &nbsp; | 100% | 0 | 0 |
&nbsp; | 100%
Total: Total | &nbsp; | 94% | 1834 | 102 | 58 | &nbsp; | 87% | 280 | 25 | &nbsp; | 93%
```

12.2. Поріг покриття у pyproject.toml

```
[tool.coverage.run]
source = ["student_manager"]
branch = true
omit = [
    "*/__main__.py", # демонстраційні сценарії лабораторних
    "*/main.py",     # консольна точка входу ЛР1
    "*/cli.py",
    "*/benchmark.py", # експерименти з часом і пам'яттю
]

[tool.coverage.report]
fail_under = 80
show_missing = true
skip_empty = true
exclude_also = [
    "if TYPE_CHECKING:",
    "if __name__ == '__main__':",
    "^\\s*\\.\\.\\.\\.\\.\\$",
]
```

12.3. Недостатньо протестовані шляхи

- streaming/experiment.py — 71 %: функції, що друкують таблиці експерименту ЛР3 і ганяють 500 000 рядків; їх свідомо не запускають у тестах, вони перевірені вручну під час експерименту.

- `registry.py` — 80 %: `search()` із кількома критеріями та `statistics()` не мають прямих тестів; чотири частково пройдені гілки (`BrPart` = 4) — комбінації фільтрів, які варто параметризувати.
- `storage/exporters.py` — 87 %: гілки `backup=True` для CSV та YAML-експортерів не покриті; JSON-експортер покритий повністю.
- `storage/readers.py` — 88 %: обробка некоректного JSON/YAML (не список, не словник) — сценарії, які інтеграційні тести не створюють.

13. Знайдені дефекти

Дефектів у продакшн-коді нові тести не виявили: перевірка `nan`, обох меж і винятків підтвердила, що модель ЛР1 поводить себе правильно (`not 0 <= nan <= 100` дає `True`, тому `nan` відхиляється). Один дефект знайдено в самому наборі тестів: перша версія `autouse`-фікстури піднімала рівень логера `student_manager` до `WARNING` «щоб не шуміло», і тест `LP5 test_logged_operation_logs_success_and_failure`, який читає `INFO` через `caplog`, упав. `Autouse`-фікстура впливає на кожен тест, тож не повинна змінювати поведінку, яку хтось перевіряє. Її замінено на сторож, що лише перевіряє відсутність витоків стану.

14. Перевірка типів і Git

```
$ mypy src
Success: no issues found in 56 source files

$ git log --oneline -4
fe1755b lab6: pytest markers, branch coverage with 80% threshold, README section, version 0.6.0
30d4fe1 lab6: restructure tests into conftest + unit/ + integration/, add fixtures, parametrize, mocks, AsyncMock
15a4cc9 lab6: clock/snapshot/notify seams for testability
2307033 Merge branch 'lab5'

$ git status --short
(чисто)
```

15. Відповіді на контрольні запитання

1. Що таке regression?

Поломка того, що раніше працювало, після нової зміни; автоматичні тести ловлять її до коміту.

2. Чим integration test відрізняється від unit test?

Unit перевіряє один модуль ізольовано і швидко; integration — кілька модулів разом із реальними файлами чи конфігурацією.

3. Що показує test pyramid?

Unit-тестів має бути найбільше, інтеграційних менше, end-to-end найменше: чим вище, тим повільніше й дорожче.

4. Як pytest знаходить tests?

Файли `test_*.py` у `testpaths`, у них функції `test_*` і класи `Test*`.

5. Що означає Arrange–Act–Assert?

Підготувати дані, виконати дію, перевірити результат — три частини кожного тесту.

6. Чому floats не завжди слід порівнювати через `==`?

Двійкове подання неточне: $(70.1+70.2+70.3)/3 \neq 70.2$; тому `pytest.approx`.

7. Як отримати exception object через `exc_info`?

`with pytest.raises(E) as info:` — потім `info.value`, `info.type`, `info.value.__cause__`.

8. Які boundaries потрібно тестувати для діапазону 0–100?

0, 100, значення одразу за ними (-0.01 , 100.01) і нечислові випадки на кшталт `nan`.

9. Що дають ids параметризованих tests?

Читабельні імена у виводі (`[nan]`, `[strict]`) замість `[param0]`, і можливість запустити один випадок через `::test[id]`.

10. Які fixture scopes підтримує pytest? Який default?

`function` (default), `class`, `module`, `package`, `session`.

11. Для чого yield у fixture?

Код до `yield` — `setup`, після — `teardown`, який виконується навіть після падіння тесту.

12. Чому fixtures із `conftest.py` не імпортують вручну?

`pytest` реєструє їх як плагін каталогу і впроваджує за іменем параметра; імпорт зламав би область дії та порядок.

13. Як monkeypatch змінює environment variable?

`monkeypatch.setenv("X", "1") / delenv("X", raising=False)`; після тесту значення повертається.

14. Чим Stub, Fake, Mock і Spy відрізняються?

`Stub` повертає заготовлені відповіді; `Fake` — робоча спрощена реалізація (in-memory репозиторій); `Mock` перевіряє взаємодію; `Spy` записує виклики, делегуючи справжньому об'єкту.

15. Що таке `spec` і для чого?

Обмежує дублер атрибутами справжнього класу: опечатка в імені методу стає `AttributeError`, а `autospec` ще й перевіряє сигнатуру.

16. Чим `return_value` відрізняється від `side_effect`?

`return_value` — одна відповідь завжди; `side_effect` — функція, виняток або послідовність відповідей.

17. Як перевірити, що `method` викликано один раз / не викликався?

`assert_called_once_with(...)` та `assert_not_called()`; для корутин — `assert_awaited_once_with`, `assert_not_awaited`.

18. Чому `patch` потрібно виконувати там, де `name` використовується?

`from m import f` копіює посилання в модуль-споживач; підміна `m.f` його не зачепить, тому патчити треба `consumer.f` або звертатися через модуль, як `clock.now`.

19. Що таке `flaky test` і чому він небезпечний?

Тест, який то проходить, то ні без змін у коді (час, мережа, порядок). Команда починає ігнорувати червоний прогін і пропускає справжні поломки.

20. Чим `statement coverage` відрізняється від `branch coverage`?

`Statement` рахує виконані рядки; `branch` — чи пройдено обидва виходи кожного `if`/циклу. Рядок може бути виконаний, а гілка `else` — ні.

21. Чи гарантує `100% coverage` відсутність `bugs`?

Ні: `coverage` показує, що код виконувався, а не що результат перевірено; тест без `assert` дає 100 % і нічого не ловить.

22. Що таке `coverage threshold`?

Мінімальний відсоток (`fail_under = 80`), нижче якого прогін вважається невдалим — запобіжник від тихого падіння покриття.

23. Чому `tests` повинні бути незалежними?

Щоб їх можна було запускати в будь-якому порядку і по одному; спільний стан робить падіння залежним від сусідів — саме це ловить `isolation_guard`.

16. Висновок

Набір тестів проєкту переструктуровано у `conftest.py`, `unit/` та `integration/` і розширено з 118 до 175 тестів; усі проходять за ~1 с, муру у `strict`-режимі не має зауважень. Покриття зі

гілками — 93 % при порозі 80 %, закріпленому в `pyproject.toml`. Найкорисніші прийоми: параметризована фікстура, яка подвоює інтеграційний тест без копіювання коду; `create_autospec`, що ловить неправильні виклики дублера; `AsyncMock` без додаткових плагінів; і один модуль `clock`, завдяки якому час у проєкті детермінований. Головний урок — `autouse`-фікстура не повинна змінювати поведінку, яку перевіряє інший тест.